Observability in Kubernetes

chukmunnlee@nus.edu.sg

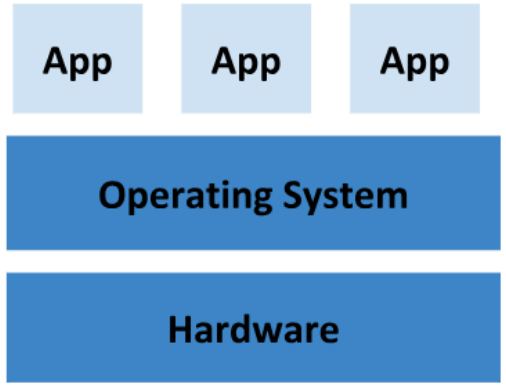
© Copyright National University of Singapore

Kubernetes Review

© Copyright National University of Singapore

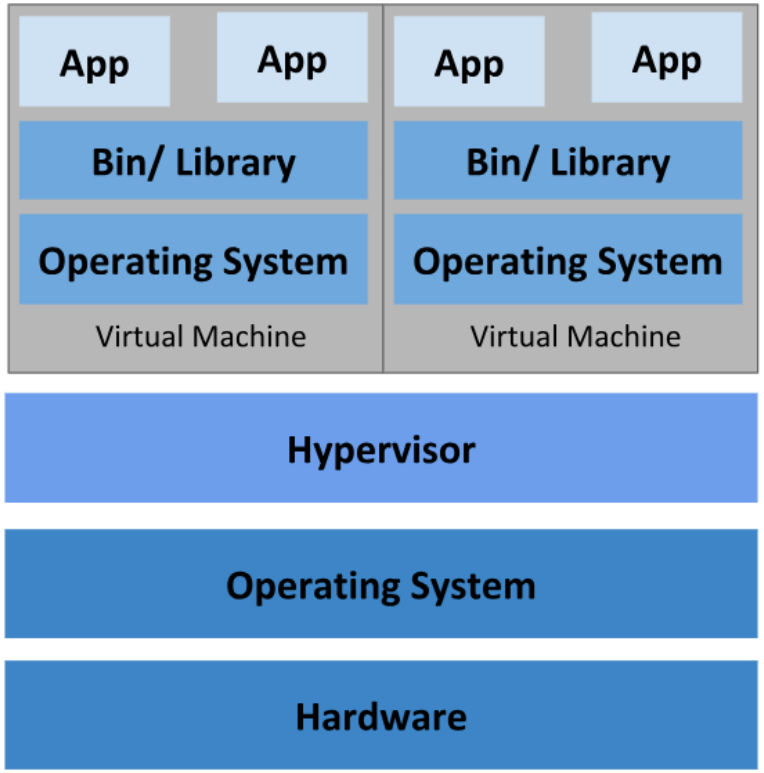
Deployment Patterns

Applications were deployed on physical servers



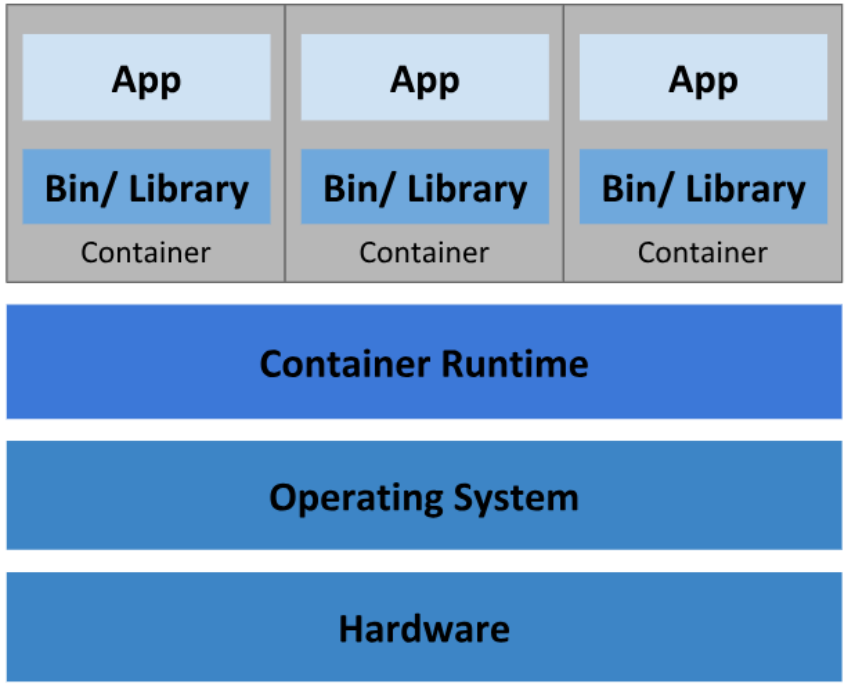
Traditional Deployment

Virtualization allows multiple VMs to run on a single physical server. Reduce cost and complexity.



Virtualized Deployment

Containers are similar to VMs viz. provide resource isolation but are lightweight.



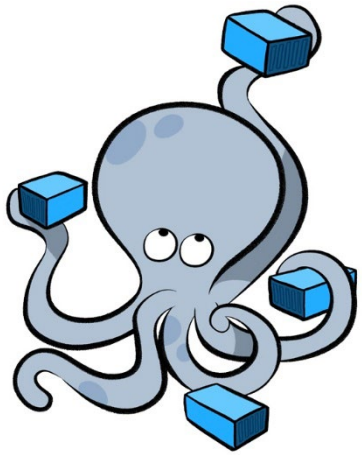
Container Deployment

From https://kubernetes.io/docs/concepts/overview/

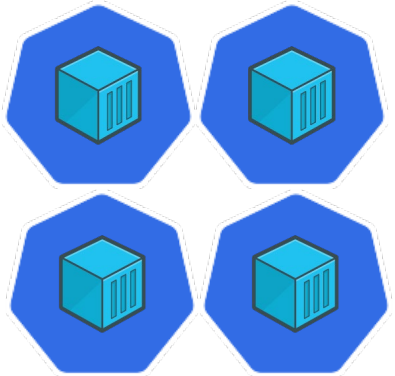
© Copyright National University of Singapore

Container Deployment Strategies

Docker Compose - Multiple cooperating containers

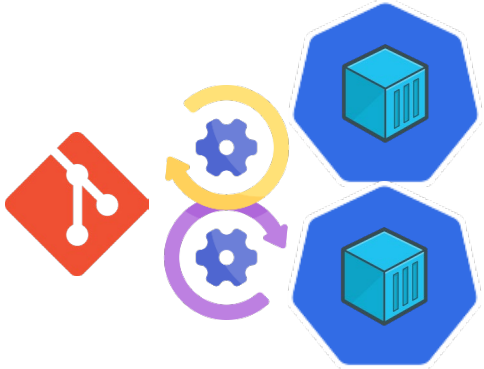


Docker - Single container



Kubernetes - Managed applications

GitOps - Managed applications and infrastructure



© Copyright National University of Singapore

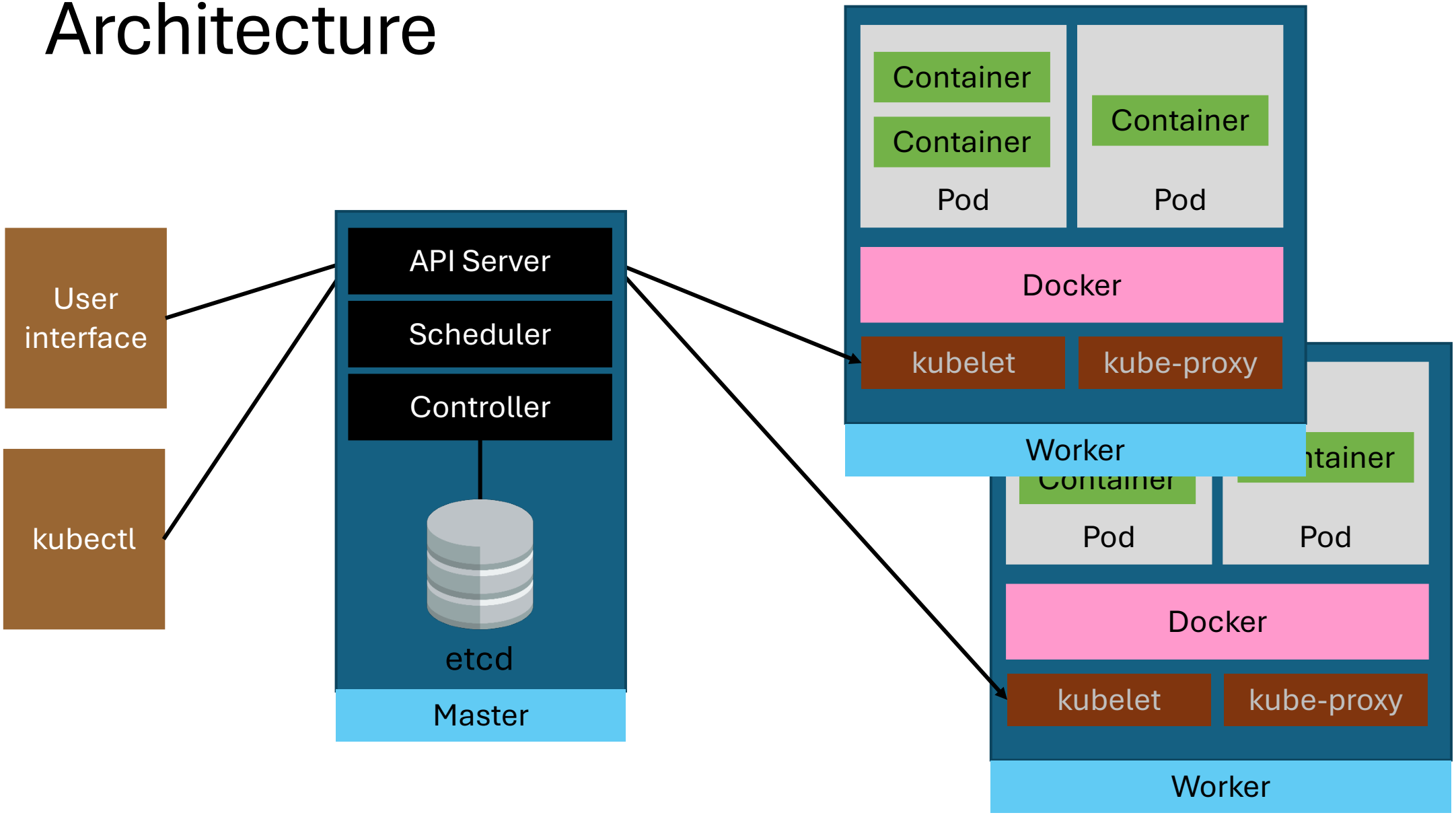
Kubernetes Overview

- Kubernetes is a container orchestration platform
  - Ensures that applications deployed as containers does not go down
  - Eg. containers are restarted when the primary process exits
- Key features
  - Self healing - restarts containers when they fail or fail health checks
  - Configuration and secrets management - manages sensitive data eg password, API keys and configuration data
  - Service discovery and load balancing - uses DNS for service lookup. Request are distributed a cross replicas
  - Automated rollouts and rollbacks - application updates are rollout progressively to avoid disrupting service availability
  - Storage orchestration - automatically provision and mount volumes into containers
  - Horizontal scaling - applications can be scaled up or down based of resource or application metrics

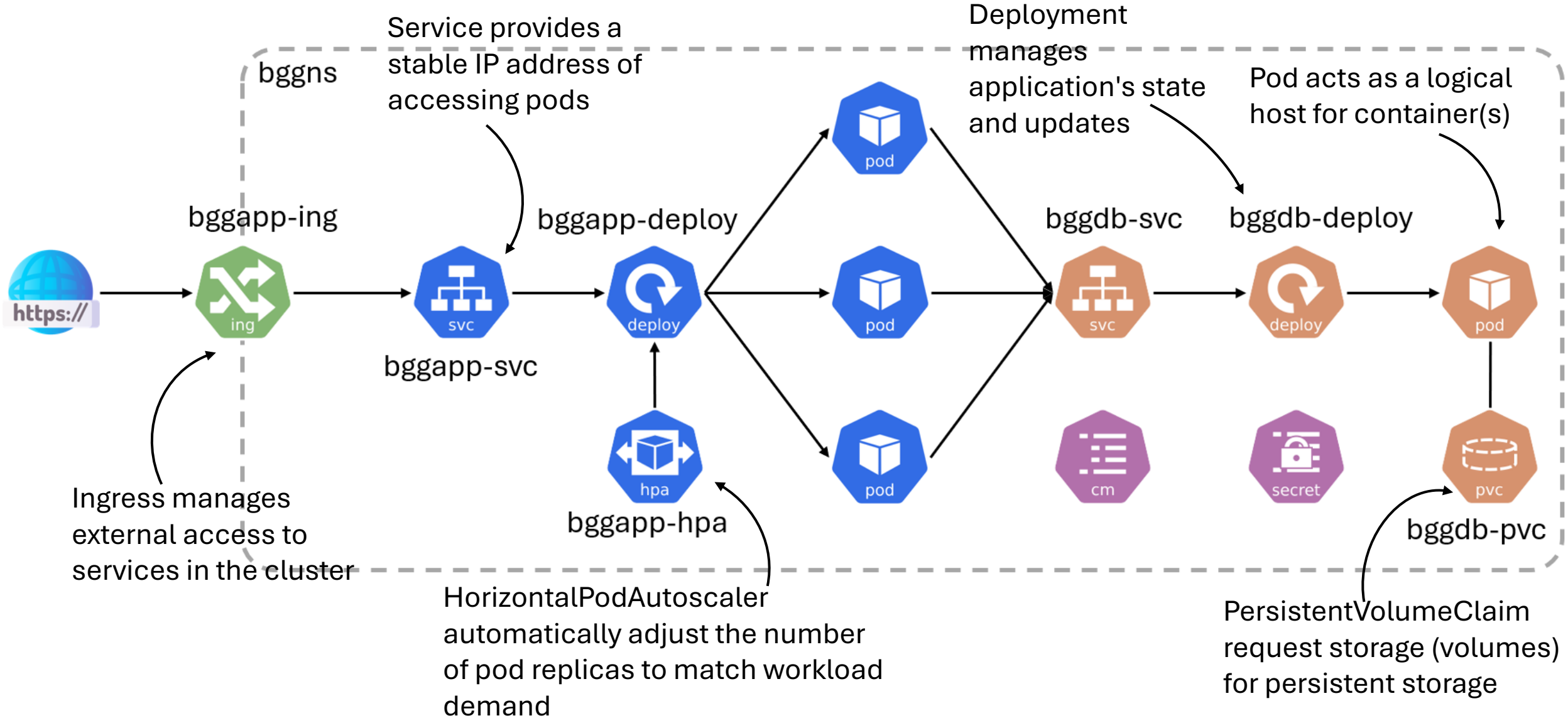
From https://kubernetes.io/docs/concepts/overview/#why-you-need-kubernetes-and-what-can-it-do

© Copyright National University of Singapore

Architecture



Typical Kubernetes Deployment



© Copyright National University of Singapore

Workshop 4
Task 1

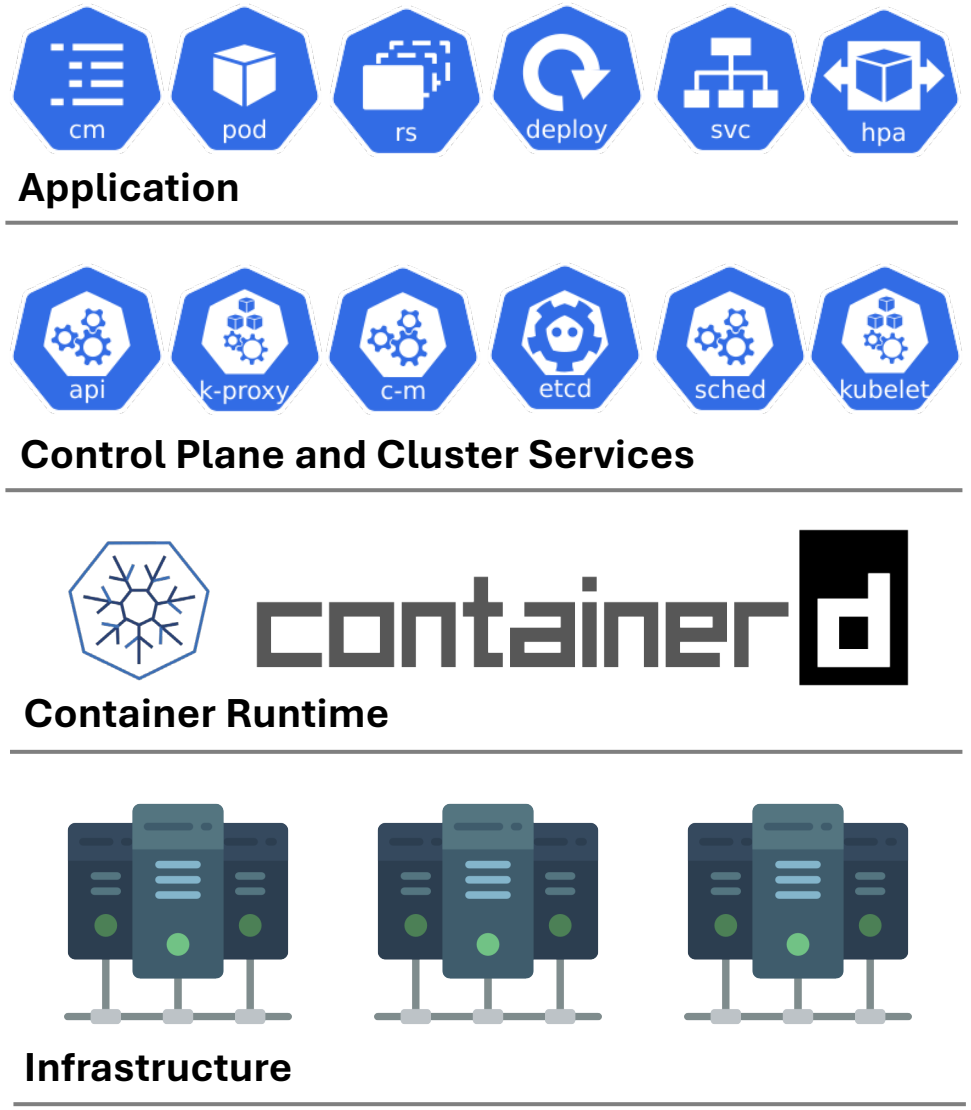
© Copyright National University of Singapore

Observability in Kubernetes

© Copyright National University of Singapore

Observability in Kubernetes is Challenging

- Pods/containers are distributed and ephemeral
  - Pods can be dynamically rescheduled making them difficult to track
- Multiple layers of abstraction
  - Kubernetes layers include application, container runtime, networking, orchestration, operation system
- Logs explosion
  - Each layer can generate lots of data, filtering and understanding them requires expertise



© Copyright National University of Singapore

Default Telemetry Signals from Kubernetes

- Health checks - monitors pod's health
  - Liveness, readiness and startup probes for containers
  - Node conditions provide the health status of the nodes eg Ready, MemoryPressure, etc

```
kubectl get node node-a -ojsonpath='{.status.conditions[*].status}'
```

  - Node are automatically tainted/untainted when node condition changes
    - Eg. Node's free memory is low, (like) adds then following taint

```
kubect taint node-a node.kubernetes.io/disk-pressure:NoSchedule
```

- Logs - multiple sources including
  - Container and system component logs

```
kubectl logs deployment/myapp -nmyns
```

  - Event logs - describes resources' lifecycle

```
kubectl describe po/mypod -nmyns
kubect get events -nmyns
```

© Copyright National University of Singapore

Default Telemetry Signals from Kubernetes

- Audit logs - tracks API server requests
  - https://kubernetes.io/docs/tasks/debug/debug-cluster/audit/
  - Disabled by default
- Tracing - only for API and Kubelet, currently in beta
  - Traces have to be forwarded to a OTLP collector
  - https://kubernetes.io/docs/concepts/cluster-administration/system-traces
  - Disabled by default
- Metrics - only provides component's health and container metrics
  - cAdivsor is part of the Kubelet, exposes CPU, memory, disk/network I/O
    - Used by Prometheus
  - Require to install metrics server for resource metrics: CPU and memory utilization

    ```
    kubectl top nodes
    ```
  - https://github.com/kubernetes-sigs/metrics-server

© Copyright National University of Singapore

Kubernetes Telemetry

- OpenTelemetry has a set of specialized receivers that scrapes metrics and logs from nodes, control plane components and Kubernetes objects eg deployment
- Some of these receivers have to run on the node
  - To collect data specific to the node

© Copyright National University of Singapore

Kubernetes Specific Receivers and Processors

- Infrastructure receivers - node specific
  - `hostmetrics` (metrics) - collects node level metrics eg. CPU, disk memory, networking, etc from `/proc` and `/sys`
  - `k8slog` (logs) - a specialized `filelog` receiver that only scrapes Kubernetes logs from `/var/log/pods`
  - `kubeletstats` (metrics) - gather container and pod metrics from kubelet eg. CPU, memory, filesystem etc for pods and containers running on that node
- Cluster receivers
  - `k8s_cluster` (logs) - provide a high level view of the health of the Kubernetes cluster reporting on disk pressure, memory allocations, etc.
  - `k8s_events` (logs) - cluster wide events like pod scheduling, image pull status, liveness probes, OOM, etc
  - `k8sobjects` (logs) - reports changes on any resource objects like state of all the pods, what is running

© Copyright National University of Singapore

Receivers

Cluster wide

Application

Control Plane and Cluster Services

Per node

containerd

Container Runtime

Infrastructure

receivers	exporters
otlp	**metrics** hostmetrics k8s_cluster kubeletstats otlp
k8s_cluster k8s_events k8sobjects	**traces** otlp
kubeletstats k8slog	**logs** k8slogs k8s_events k8sobjects filelog otlp
hostmetrics filelog	

© Copyright National University of Singapore

Example - **hostmetrics**

```
receivers:
  hostmetrics:
    collection_interval: 10s
    root_path: /hostfs
    scrapers:
      cpu: {}
      memory: {}
      load:
        cpu_average: true

exporters:
  otlp/metrics:
    endpoint: ...

service:
  pipelines:
    metrics:
      receivers: [hostmetrics]
      exporters: [otlp/metrics]
```

https://github.com/open-telemetry/opentelemetry-collector-contrib/tree/main/receiver/hostmetricsreceiver

© Copyright National University of Singapore

Example - **kubeletstats**

```
receivers:
  kubeletstats:
    auth_type: serviceAccount
    collection_interval: 20s
    endpoint: ${env:K8S_NODE_IP}:10250
    metric_groups: [ pod, container ]

...
exporters:
  otlp/metrics:
    endpoint: ...

service:
  pipelines:
    metrics:
      receivers: [kubeletstats]
      processors: [batch]
      exporters: [otlp/metrics]
```

https://github.com/open-telemetry/opentelemetry-collector-contrib/tree/main/receiver/hostmetricsreceiver

© Copyright National University of Singapore

Example - k8slog

```
receivers:
  k8slog:
    discovery:
      mode: daemonset-stdout
      host_root: /hostfs
      filter:
        namespaces:
        - op: equals
          value: books-ns
        containers:
        - op: not-equals
          value: otc-container
    extract:
      metadata:
      - k8s.pod.name
      - k8s.namespace.name
      - k8s.node.name
      - k8s.deployment.name
    operators:
    - type: recombine
      combine_field: body
      is_first_entry: body matches "^\\d{4}-\\d{2}-\\d{2}"
      max_log_size: 128kb
      source_identifier: attributes["k8s.pod.uid"]
```

https://github.com/open-telemetry/opentelemetry-collector-contrib/tree/main/receiver/k8slogreceiver

© Copyright National University of Singapore

Example - k8s_cluster

```
receivers:
  k8s_cluster:
    auth_type: serviceAccount
    k8s_leader_elector: k8s_leader_elector
    collection_intervals: 30s
    node_conditions_to_report: [ Ready, MemoryPressure, DiskPressure ]
    allocatable_types_to_report: [ cpu, memory, pods ]
...
exporters:
  otlp/metrics:
    endpoint: ...

service:
  pipelines:
    metrics:
      receivers: [k8s_cluster]
      processors: [batch]
      exporters: [otlp/metrics]
```

https://github.com/open-telemetry/opentelemetry-collector-contrib/tree/main/receiver/k8sclusterreceiver

© Copyright National University of Singapore

Example - k8sobjects

```
receivers:
  k8sobjects:
    auth_type: serviceAccount
    objects:
    - name: events
      group: events.k8s.io
      mode: watch
      namespaces: [ prod, staging ]
    - name: pods
      mode: pull
      interval: 30s
      namespaces: [ prod ]
      label_selector: app=critical
...

service:
  pipelines:
    logs:
      receivers: [k8s_cluster]
      processors: [batch]
      exporters: [otlp/logs]
```

https://github.com/open-telemetry/opentelemetry-collector-contrib/tree/main/receiver/k8sobjectsreceiver

© Copyright National University of Singapore

Kubernetes Metadata

© Copyright National University of Singapore

Enriching Signals

- Integrate telemetry data from services running on Kubernetes
  - Eg. syslog, legacy applications, IoT gateway, etc.
  - Enrich these data with 'who', 'what' and 'where'
- `k8sattribute` process bridges these signals with OpenTelemetry
  - Receivers ingest raw data with filelog, syslog
  - Queries the API server to identify and enrich the signals
    - Eg. What pod has the IP address 10.10.0.55
    - Eg. What is the name of the container with id abc123
  - Exports the enriched data to backends where they can be correlated

© Copyright National University of Singapore

`k8sattributes` Processor

- **`k8sattributes`** processor automatically injects relevant metadata into spans, metrics and logs
  - Uses API server to discover pods running the cluster
  - Caches the details of the pods
  - Use by adding as a processor to OTel's telemetry pipeline
- Enriches the signals by looking up relevant pod details from the cache
  - Associates the signals-to-pod association by examining specific attributes from signals
- Uses a combination of one of the following to match a pod uniquely
  - Pod IP
    - **`k8s.pod.ip`**
  - Pod UID
    - **`k8s.pod.uid`**
  - Pod namespace/pod name
    - **`k8s.namespace.name`**
    - **`k8s.pod.name`**

© Copyright National University of Singapore

Example - Enriching syslog logs

```yaml
receivers:
  syslog/tcp:
    tcp:
      endpoint: 0.0.0.0:5452
    protocol: rfc5424
  syslog/udp:
    udp:
      endpoint: 0.0.0.0:5452
    protocol: rfc5424
```

```yaml
service:
  pipelines:
    logs/syslog:
      receivers: [syslog]
      processors: [k8sattributes, batch]
      exporters: [loki]
```

```yaml
processors:
  k8sattributes:
    auth_type: "serviceAccount"
    passthrough: false
    pod_association:
    - sources:
      - from: connection
    extract:
      metadata:
      - k8s.pod.name
      - k8s.namespace.name
      - k8s.deployment.name
      labels:
      - tag_name: service.version
        key: app-version
        from: pod
```

Lookup the API server

Telemetry data to pod association rule. Use the IP address

Types of information to extract from the associated pod

Enrich the signal with these data from the pod

© Copyright National University of Singapore

Pod Association and Data Extraction

- Pod association attempt to match the signal with the pod that generated the signal
- Rules
  - Match the pod ip, `k8s.pod.ip`, from the signal's resource attribute
  - Match the pod uid, `k8s.pod.uid`, from the signal's resource attribute
  - If the above 2 is not present then use the IP from the incoming connection
- Extract specifies what metadata to extract and enrich the signal
- Sources of data
  - pod itself e.g. namespace, pod name, IP, node name, deployment name, etc
  - labels and annotations
    - From the pod, deployment/statefulset/replicaset, namespace
    - Environment variable from the pod

See https://github.com/open-telemetry/opentelemetry-collector-contrib/tree/main/processor/k8sattributesprocessor#complete-configuration-options

© Copyright National University of Singapore

Example - Enrich Application Log File

```
{"timestamp": "2026-...Z", "level": "info", "msg": "User login successful", "trace_id":
"4bf9...", "span_id": "00f0...", "pod_uid": "550e8...", "user_id": "user_99", "service":
"authenticate" }
```

```
receivers:
  filelog:
    include: [/app/audit.log]
    operators:
    - type: json_parser
    - type: trace_parser
      trace_id:
        parse_from: attributes.trace_id
      span_id:
        parse_from: attributes.span_id
    - type: severity_parser
      parse_from: attributes.level
    - type: move
      from: attributes.pod_uid
      to: resource['k8s.pod.uid']
    - type: move
      from: attributes.service
      to: resource['service.name']
```

Identify the log line with uid, if fail use the IP address

```
processors:
  k8sattributes:
    auth_type: serviceAccount
    passthrough: true
    pod_association:
    - sources:
      - from: resource_attribute
        name: k8s.pod.uid
    - sources:
      - from: connection
    extract:
      metadata:
      - k8s.pod.name
      - k8s.namespace.name
      - k8s.deployment.name
```

```
service:
  pipelines:
    logs:
      receivers: [ filelog ]
      processors: [ k8sattributes, batch ]
      exporters: [ otlp/loki ]
```

© Copyright National University of Singapore

OpenTelemetry Operator

© Copyright National University of Singapore

OpenTelemetry Operator

- Implements the Kubernetes operator pattern to manage OTel components within a Kubernetes cluster
  - Simplifies the deployment of OTel collector with CRDs
  - Auto inject sidecars into pods to perform auto-instrumentation
- Provides the following OTel deployment modes for different architectural needs
  - Deployment - standalone services eg OTel collector in gateway mode
  - DaemonSet - run one collector per node in agent mode
    - Eg. for collecting logs from nodes
  - StatefulSet - like Deployment, but require collector identity or persistent storage
  - Sidecar - injects collector instances directly into every pod

© Copyright National University of Singapore

OTel Collector CRDs

CRD	Description
`OpenTelemetryCollector`	Defines a configuration and deployment topology for an instance of the OpenTelemetry collector. Accepts a standard OTel configuration
`Instrumentation`	Defines a configuration on how to auto-instrumentation an application. Only required if application is not instrumentd Optionally specifies the language so that the appropriate instrumentation agent can be injected into the pod

- Deploy an instance of **`OpenTelemetryCollector`** (**`otelcol`**) resource to receive telemetry data
- Define how applications send instrumentation data to the OTel collector instance by creating an **`Instrumentation`** (**`otelinst`**)
- Annotate the application's pod to reference the **`otelcol`**
  - Eg. **`instrumentation.opentelemetry.io/inject-nodejs: "<otel collector name>"`**
- The operator detects the auto instrument annotations, injects sidecar containers to instrument the application in the pod and forward telemetry data to the **`otelcol`**

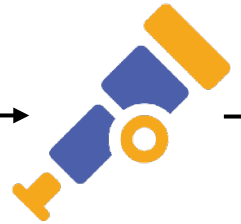
© Copyright National University of Singapore

OTel Collector CRD Architecture

CRD (otelcol)

`OpenTelemetry Collector`

1. Creates an instances of OTel collector



6. Exports telemetry data to configured backends

Optional

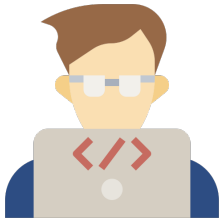
CRD (otelinst)

`Instrumentation`

2. Describe how to auto instrument application

5. Forward telemetry signals to OTel collector

3. Annotate pod
`instrumentation.opentelemetry.io/inject-nodejs: <otelinst name>`

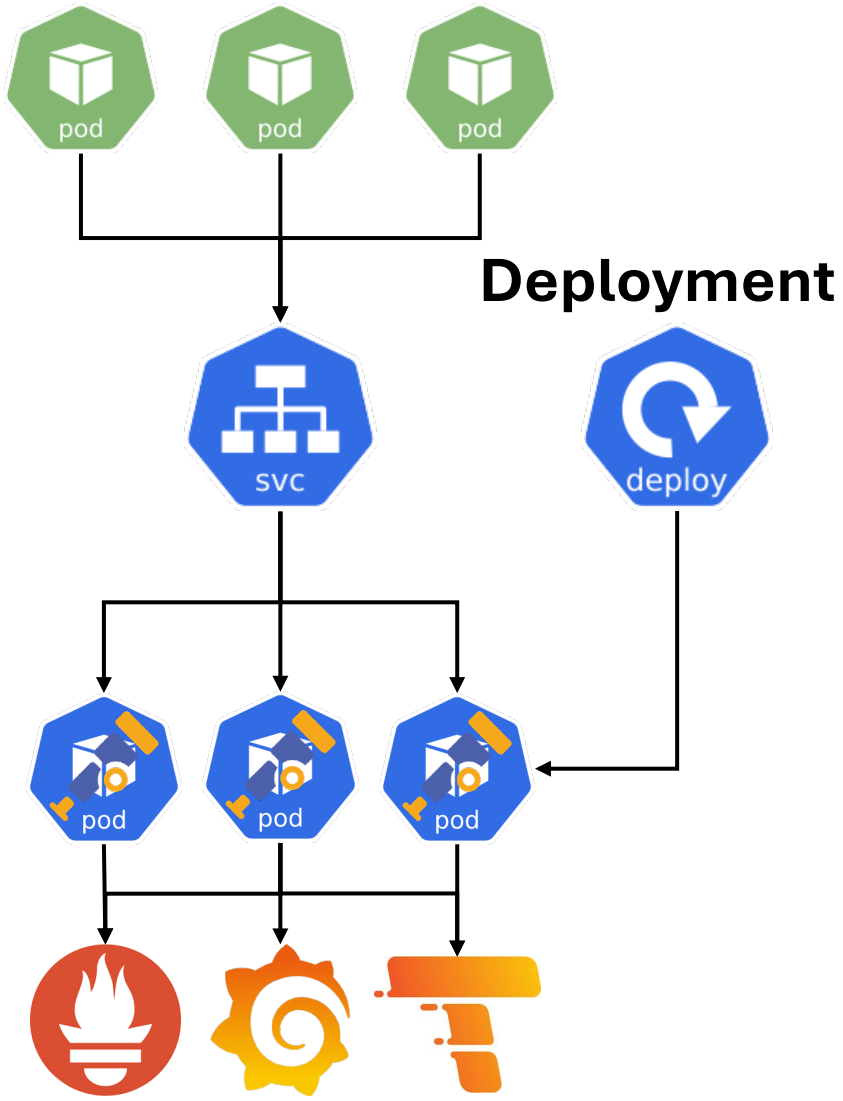


4. Deploy pod, operator injects sidecar-container into pod to auto instrumentation application

© Copyright National University of Singapore

Deployment Modes - Deployment

- Acts as a centralized gateway to process signals from pods or other collectors
  - Performs aggregation, batching, transformation, sampling, etc before exporting to backend
- Each instance of the OpenTelemetry is stateless
  - Runs in parallel
  - Can be scaled dynamically



© Copyright National University of Singapore

Example - OTel Collector

```yaml
apiVersion: opentelemetry.io/v1alpha1
kind: OpenTelementryCollector
metadata:
  name: bgg-otelcol
spec:
  mode: deployment
  image: otel/opentelemetry-collector-contrib:latest
  resources:
    requests:
      memory: 1024Mi
      cpu: 100m
    limits:
      memory: 1024Mi
      cpu: 100m
  config:
    receivers:
      ...
    service:
      ...
```

Use a **Deployment** resource for deployment

Use CRD to deploy an instance of OTel collector using the specified image

Pod's resource requirements

Valid OTel configuration. Must be supported by the image version

© Copyright National University of Singapore

Example - OTel Collector

Creates 3 services

One exposed port for each receiver

```
[test](master) k get otelinst,all -nbggns
NAME                                          READY   STATUS    RESTARTS   AGE
pod/bgg-otelcol-collector-59c956f84c-btqgp    1/1     Running   0          2d19h

NAME                                       TYPE        CLUSTER-IP      EXTERNAL-IP   PORT(S)                      AGE
service/bgg-otelcol-collector              ClusterIP   10.99.146.203   <none>        4317/TCP,4318/TCP,9090/TCP   2d19h
service/bgg-otelcol-collector-headless     ClusterIP   None            <none>        4317/TCP,4318/TCP,9090/TCP   2d19h
service/bgg-otelcol-collector-monitoring   ClusterIP   10.111.139.11   <none>        8888/TCP                     2d19h

NAME                                     READY   UP-TO-DATE   AVAILABLE   AGE
deployment.apps/bgg-otelcol-collector    1/1     1            1           2d19h

NAME                                                DESIRED   CURRENT   READY   AGE
replicaset.apps/bgg-otelcol-collector-59c956f84c    1         1         1       2d19h
[test](master)
```

Use **`Deployment`** to rollout the OTel collector

© Copyright National University of Singapore

`otelcol` Provisioned Services

- **`bgg-otel-collector`**
  - Telemetry data ingestion service, ClusterIP
  - Exposes ports that are defined in **`receivers`** attribute

```
receivers:
  otlp:
    protocols:
      grpc:
        endpoint: 0.0.0.0:4317
      http:
        endpoint: 0.0.0.0:4318
...
exporters:
  prometheus:
    endpoint: 0.0.0.0:9090
    ...
```

Exposes the following ports

Push

Pull

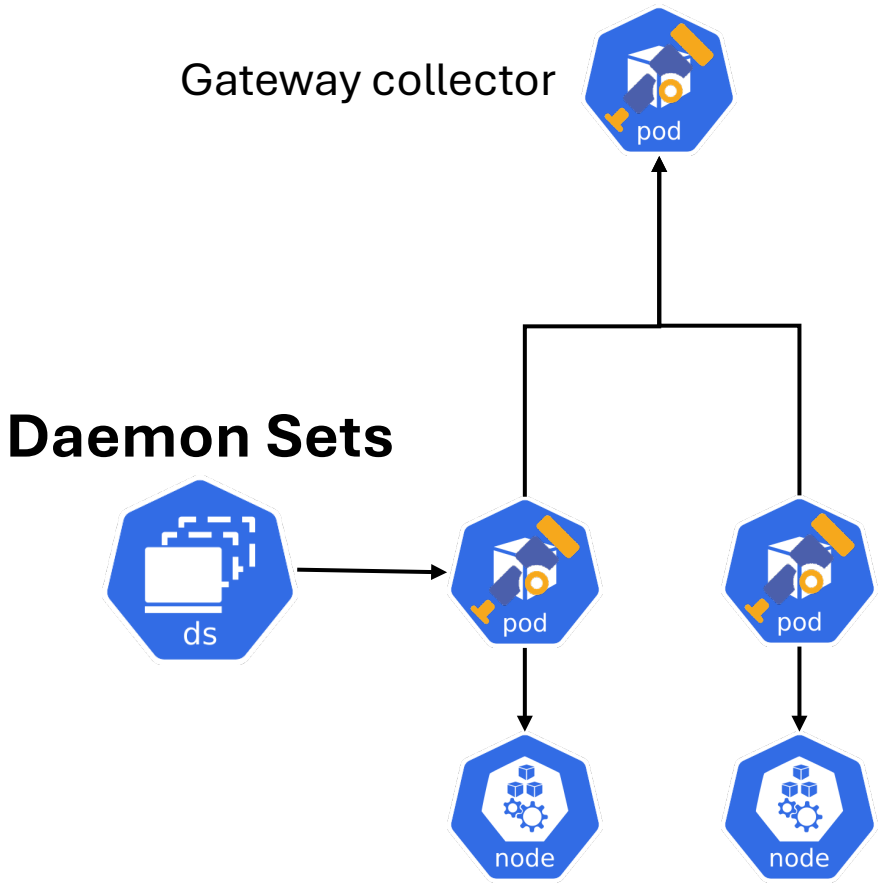
- **`bgg-otel-collector-headless`**
  - Ingestion service for **`StatefulSet`** deployment, no ClusterIP
    - Allows clients to lookup individual instance of OTel collector pods
  - Use cases
    - gRPC client side load balancing
    - Prometheus needs to scrape every OTel collector pods
- **`bgg-otel-collector-monitoring`**
  - Exposes the collectors internal telemetry
    - Eg. **`otelcol_receiver_accepted_spans`**
    - Eg. **`otelcol_process_memory_rss`**

© Copyright National University of Singapore

Workshop 4
Task 2

© Copyright National University of Singapore

Deployment Modes - DaemonSets

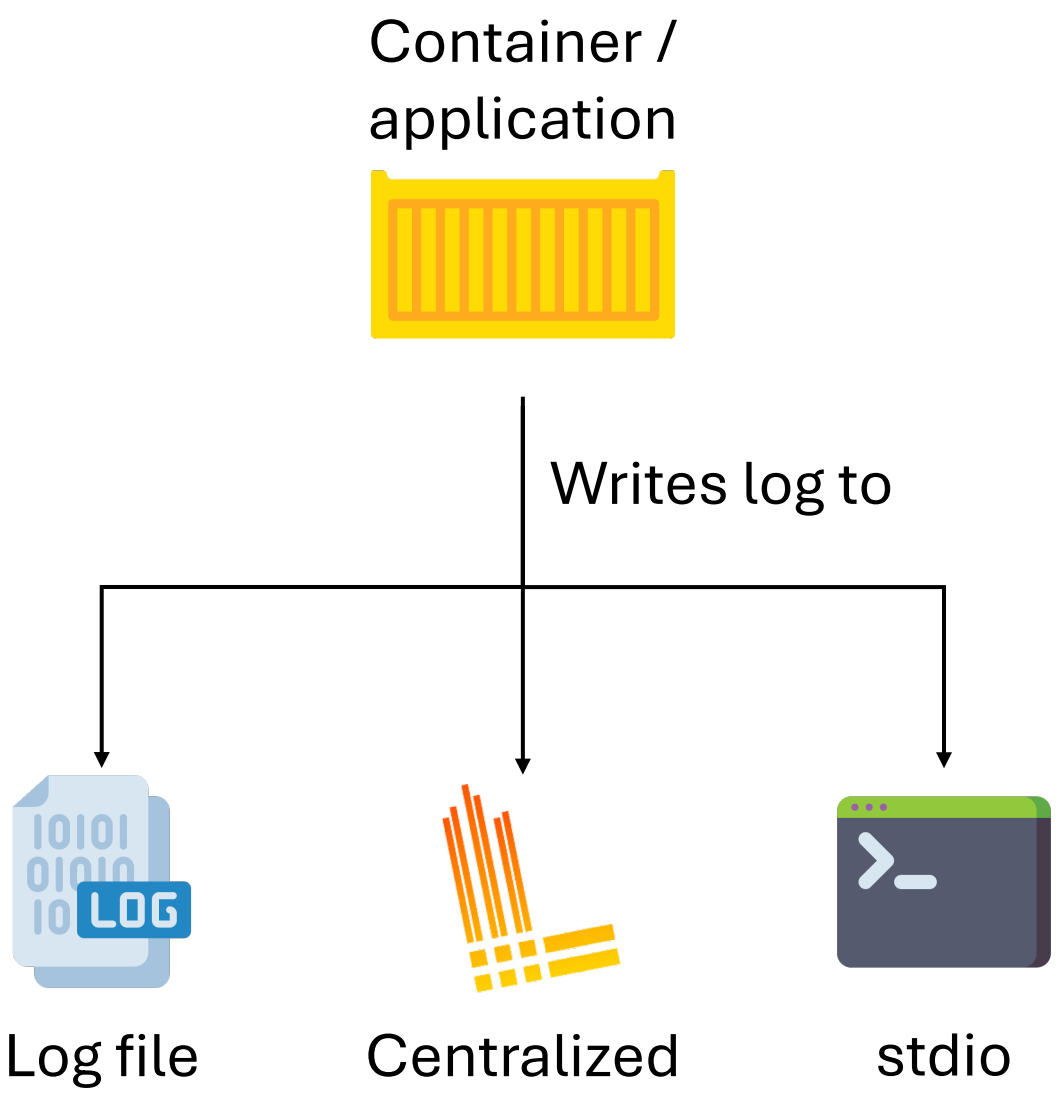


- Run an instance of the collector on every node
  - Collecting node level signals that are tied to the node e.g. log files
  - Process per node signals locally to reduce processing on the gateway
  - Acts as a buffer when the gateway is down
- Typically use with the following receivers
  - `hostmetrics`
  - `filelog`
  - `k8slog`
  - `kubeletstats`

© Copyright National University of Singapore

Container Logs

- Application logs are written to
  - Collated at a central location eg. forwarded to Loki for storage
  - Specific log file eg `/var/log/access.log`
  - Stdout / console
- Logs written to stdout are redirected by the container runtime to a file



© Copyright National University of Singapore

stdio Log Location

- Application may write logs to stdio
  - Eg. `console.info('Error')`
- Redirected by the container runtime to `/var/log/pods` directory on the worker nodes
- Uses the following directory and log structure

```
<namespace>_<pod_name>_<pod_uid>/<container_name>/0.log
```

  - `0.log` is the current active log
  - Performs log rotation or when log file reaches size limit - `1.log`, `2.log`

© Copyright National University of Singapore

CRI Log Directory Structure

```
$ sudo ls -l pods
total 124
drwxr-xr-x 3 root root 4096 Nov 28 04:48 bggns_bgg-otelcol-collector-59c956f84c-btqgp_c9b8e65b-a602-4bed-9d01-cc50cfb631ba
drwxr-xr-x 4 root root 4096 Dec  2 02:04 bggns_bggapp-deploy-68bbf58849-78vvb_affa671b-c7ef-4c24-98d6-da340ce8cdf9
drwxr-xr-x 4 root root 4096 Dec  2 02:04 bggns_bggapp-deploy-68bbf58849-d6nb7_f2d573e6-831b-4d77-ae98-a6effda0df78
drwxr-xr-x 4 root root 4096 Dec  2 02:04 bggns_bggapp-deploy-68bbf58849-h9bw9_b77f5ec1-e1d7-4e05-b812-53d4d20b37c5
drwxr-xr-x 3 root root 4096 Dec  1 01:07 bggns_bggdb-sts-0_39dd7005-4399-4d20-a11e-e0275d8ce50e
drwxr-xr-x 3 root root 4096 Nov 27 23:37 cert-manager_cert-manager-69fd4bc5fc-sjs5k_f0ebfa8a-67ca-4ed3-b07d-f663af350863
drwxr-xr-x 3 root root 4096 Nov 27 23:37 cert-manager_cert-manager-cainjector-85b6d7fc67-fqvq6_61ffb7ac-b12e-4648-9f82-0263942caa6d
drwxr-xr-x 3 root root 4096 Nov 27 23:37 cert-manager_cert-manager-webhook-cfbc49fc8-bxgsp_41c10ee0-6f93-4280-9745-af3de0c8d349
drwxr-xr-x 3 root root 4096 Nov 27 23:29 kube-system_coredns-66bc5c9577-67tjg_e540f39f-202c-4896-95d7-ba21b2be9be0
drwxr-xr-x 3 root root 4096 Nov 27 23:29 kube-system_csi-hostpath-attacher-0_0b8f287d-e402-40ce-aa23-a27afd81c369
drwxr-xr-x 3 root root 4096 Nov 27 23:29 kube-system_csi-hostpath-resizer-0_6f2da5a7-30ca-4794-a83e-3ab017f7ec57
drwxr-xr-x 8 root root 4096 Nov 27 23:30 kube-system_csi-hostpathplugin-sgmnn_f96ce35b-d6a7-4338-9283-34aed8a3badc
```

```
$ sudo ls -Rl pods/bggns_bggapp-deploy-68bbf58849-78vvb_affa671b-c7ef-4c24-98d6-da340ce8cdf9
pods/bggns_bggapp-deploy-68bbf58849-78vvb_affa671b-c7ef-4c24-98d6-da340ce8cdf9:
total 8
drwxr-xr-x 2 root root 4096 Dec  2 02:04 bggapp-container
drwxr-xr-x 2 root root 4096 Dec  2 02:04 opentelemetry-auto-instrumentation-nodejs

pods/bggns_bggapp-deploy-68bbf58849-78vvb_affa671b-c7ef-4c24-98d6-da340ce8cdf9/bggapp-container:
total 4
lrwxrwxrwx 1 root root 165 Dec  2 02:04 0.log -> /var/lib/docker/containers/6aae9ecb7a9bcd791e7d53edacdb82be37ecebac0c0747d78400d37d1b1deef9/6aae9ecb7a9bcd791e7d53edacdb82be37ecebac0c0747d78400d37d1b1deef9-json.l
og

pods/bggns_bggapp-deploy-68bbf58849-78vvb_affa671b-c7ef-4c24-98d6-da340ce8cdf9/opentelemetry-auto-instrumentation-nodejs:
total 4
lrwxrwxrwx 1 root root 165 Dec  2 02:04 0.log -> /var/lib/docker/containers/3a6d0fd9a2b8bd6d459441d259dcf34cbf9a3980d9f641c305729b454c95300a/3a6d0fd9a2b8bd6d459441d259dcf34cbf9a3980d9f641c305729b454c95300a-json.l
og
```

© Copyright National University of Singapore

Example - Scraping Container Logs

```yaml
apiVersion: opentelemetry.io/v1beta
kind: OpenTelemetryCollector
metadata:
  name: log-otelcol
spec:
  mode: daemonset
  image: otel/opentelemetry-collector-contrib
  volumes:
  - name: root-vol
    hostPath:
      path: /
      type: Directory
  volumeMounts:
  - name: root-vol
    mountPath: /hostfs
    readOnly: true
  config:
  ...
```

Mount the nodes filesystem into the collector pod

```yaml
config:
  receivers:
    k8slog:
      discovery:
        mode: daemonset-stdout
        host_root: /hostfs
  processors:
    k8sattributes:
      auth_type: serviceAccount
      passthrough: false
      pod_association:
      - sources:
        - from: resource_attribute
          name: k8s.pod.ip
      extract:
        metadata:
        - k8s.pod.name
        - k8s.namespace.name
        labels:
        - tag_name: service.name
          key: app
...
```

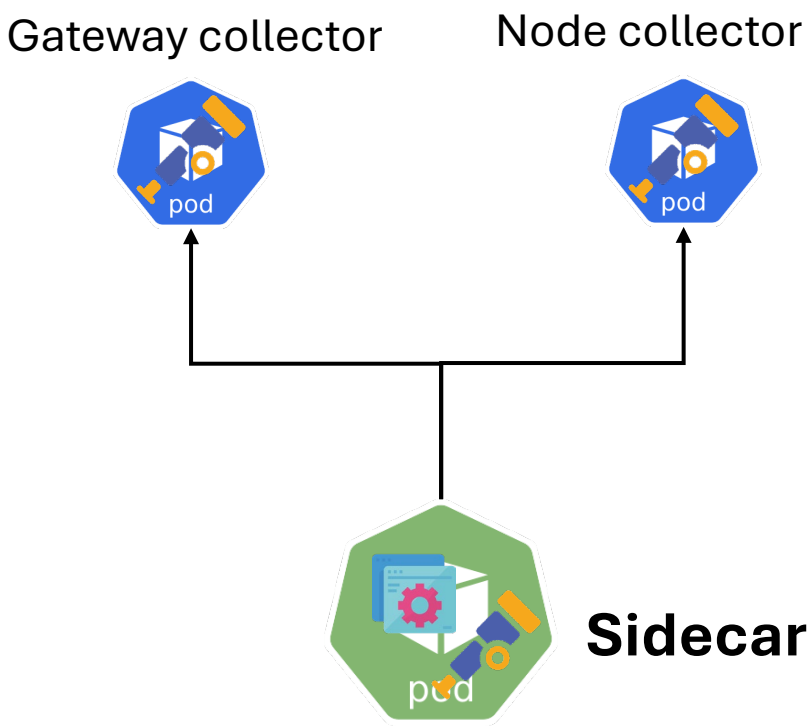
© Copyright National University of Singapore

Workshop 4
Task 3

© Copyright National University of Singapore

Deployment Modes - Sidecar

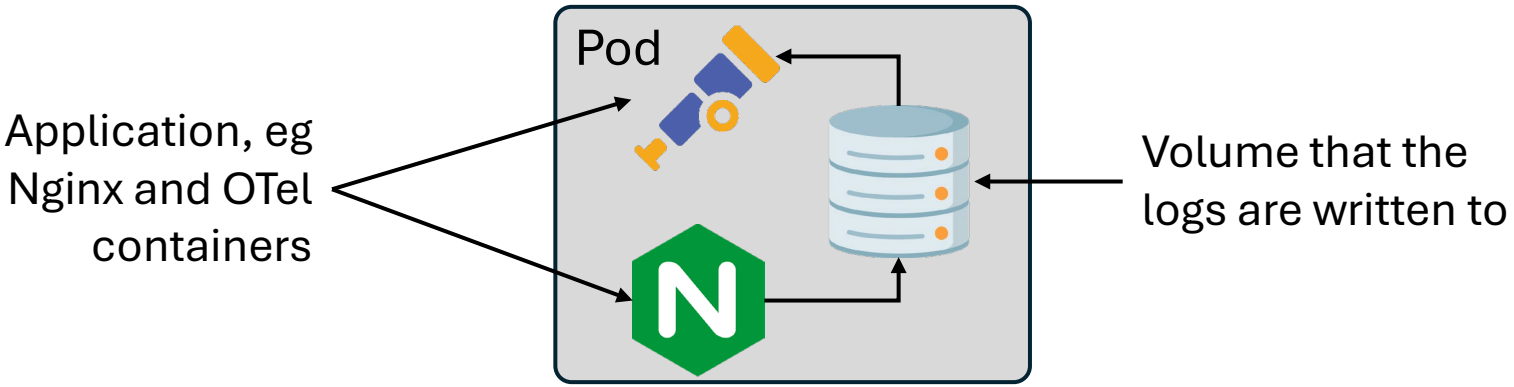
- Run an instance of the collector inside the pod as a sidecar container
  - Separate collector for each application instance
    - Collectors are isolated from other collectors
- For application specific processing
  - Eg. perform redaction before forwarding to gateway
  - Eg. application write logs to volumes mounted inside the pod



© Copyright National University of Singapore

Using Sidecar Mode

- When logs are written to volumes mounted into the pod
  - Eg. Nginx, MySQL
- OTel collector run as sidecar in the target application pod
  - Use shared volume strategy to scrape logs from volume
  - Collector is injected into pod with the following annotation
    - `sidecar.opentelemetry.io/inject: <otelcol-sidecar>`



© Copyright National University of Singapore

Example - Scraping Nginx

```yaml
apiVersion: opentelemetry.io/v1beta1
kind: OpenTelemetryCollector
metadata:
  name: sidecar-otelcol
spec:
  mode: sidecar
  image:...

  volumeMounts:
  - name: nginx-log
    mountPath: /var/log/nginx
    readOnly: true

  config:
    receivers:
      filelog:
        include:
        - /var/log/nginx/*.log
        start_at: end
        operators:
        ...
```

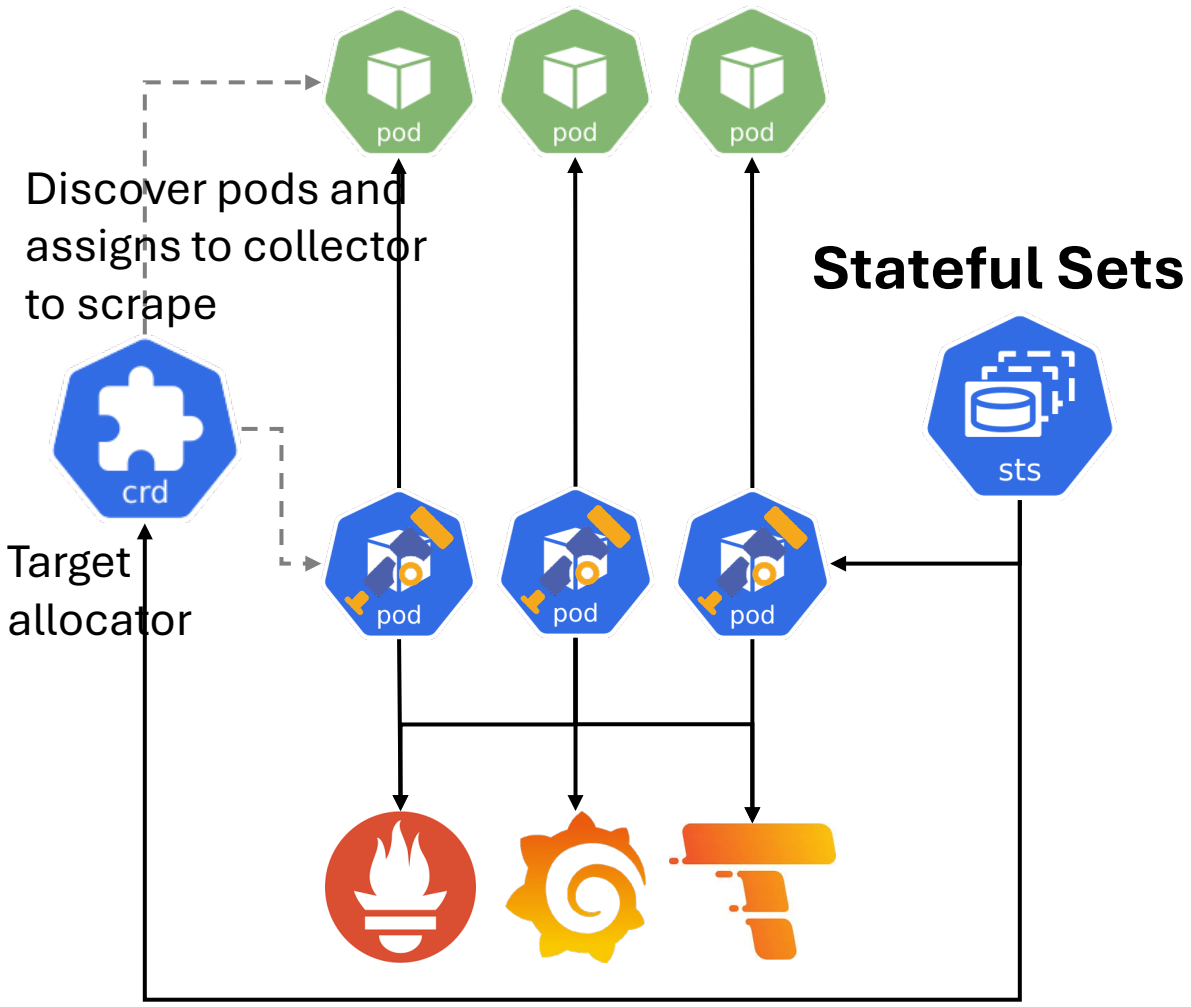
```yaml
apiVersion: v1
kind: Pod
metadata:
  name: nginx-po
  annotations:
    sidecar.opentelemetry.io/inject: sidecar-otel

spec:
  volumes:
  - name: nginx-log
    persistentVolumeClaim:
      ...
  containers:
  - name: nginx
    image: nginx
    ...
    volumeMounts:
    - name: nginx-log
      mountPath: /var/log/nginx
```

Log volume is mounted by both containers.
Volume name must match

© Copyright National University of Singapore

Deployment Modes - StatefulSets



- Like deployment mode but each collector pod has an identity
  - Commonly used with pull-based scraped targets eg Prometheus
- Issue - same application metrics can be scraped multiple times with deployment mode
- Stateful set mode can be deployed with a target allocator service to manage which collector should scrape which application pod
  - Prevent duplicate scraps
  - Integrates with Prometheus Operator
    - More later

© Copyright National University of Singapore

Auto Instrumentation

© Copyright National University of Singapore

Auto Instrumentation

- **`Instrumentation`** resource injects and configure auto-instrumentation for supported language runtime and web servers
  - Like **`getNodeAutoInstrumentations()`**
  - For traces only
- Supports the following
  - Languages - .Net, Golang, Java, NodeJS, Python
  - Web servers - Apache HTTPD, Nginx

© Copyright National University of Singapore

Example - OTel Instrumentation

Creates a resource to auto instrument (traces) applications

Use pod labels for resource attributes

Collector's OTLP endpoint

One of the services provisioned by **otelcol** resource

Inter-service propagation context

Define a trace sampler

```yaml
apiVersion: opentelemetry.io/v1beta1
kind: Instrumentation
metadata:
  name: bgg-otelinst
spec:
  defaults:
    useLabelsForResourceAttributes: true
  exporter:
    endpoint: http://bgg-otel-collector:4318
  propagators:
  - tracecontext
  - baggage
  sampler:
    type: parentbased_traceidratio
    argument: ".5"
```

© Copyright National University of Singapore

Sampling

- **`useLabelsForResourceAttributes`** adds labels as trace resource labels
  - The following labels will be scraped and added to trace
    - **`app.kubernetes.io/instance`** to **`service.name`**
    - **`app.kubernetes.io/name`** to **`service.name`**
    - **`app.kubernetes.io/version`** to **`service.version`**
- Define trace sampling on the client side
  - Set with **`sampler`** attribute
  - Valid values include **`parentbased_always_on`**, **`parentbased_always_off`**, **`parentbased_traceidratio`**,
    - See https://github.com/open-telemetry/opentelemetry-operator/blob/main/docs/api/instrumentations.md#instrumentationspecsampler

© Copyright National University of Singapore

Auto Instrumenting Pods

- Annotate pods annotation to enable instrumentation injection
  - With the name of the deployed instrumentation resource
- See for a completes list annotations
  - https://github.com/open-telemetry/opentelemetry-operator?tab=readme-ov-file#opentelemetry-auto-instrumentation-injection

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: bggapp-deploy
  annotations:
    instrumentation.opentelemetry.io/inject-
javascript: bgg-otelinst
spec:
  ...
  template:
    metadata:
      labels:
        app: bgg
    annotations:
      instrumentation.opentelemetry.io/inject-
javascript: bgg-otelinst
    spec:
      ...
```

Annotate at the pod (template) not at the deployment resource

Name of the **otelinst** resource. If **otelinst** is in another namespace precede namespace e.g. **bggns/bgg-otelinst**

© Copyright National University of Singapore

Example - Auto Injection Setup

Pod labels will be added as resource labels

```
[test](master) k describe po/bggapp-deploy-68bbf58849-78vvb -nbggns
Name:             bggapp-deploy-68bbf58849-78vvb
Namespace:        bggns
Priority:         0
Service Account:  default
Node:             minikube/192.168.39.248
Start Time:       Tue, 02 Dec 2025 10:04:32 +0800
Labels:           app=bgg
                  app.kubernetes.io/name=bggapp
                  app.kubernetes.io/version=1.0.0
                  name=bggapp-po
                  pod-template-hash=68bbf58849
Annotations:      instrumentation.opentelemetry.io/inject-nodejs: bgg-otelinst
Status:           Running
```

Annotation to auto instrument pod

© Copyright National University of Singapore

Example - Auto Injection Setup

```
Init Containers:
  opentelemetry-auto-instrumentation-nodejs:
    Container ID:  docker://89f55338b3ee6b41ad46dba2a7c27d9e282bd75a1ac75179208ef1f9199b0080
    Image:         ghcr.io/open-telemetry/opentelemetry-operator/autoinstrumentation-nodejs:0.66.0
    Image ID:      docker-pullable://ghcr.io/open-telemetry/opentelemetry-operator/autoinstrumentation-nodejs@s
    Port:          <none>
    Host Port:     <none>
    Command:
      cp
      -r
      /autoinstrumentation/.
      /otel-auto-instrumentation-nodejs
    State:          Terminated
      Reason:       Completed
      Exit Code:    0
      Started:      Mon, 01 Dec 2025 09:14:10 +0800
      Finished:     Mon, 01 Dec 2025 09:14:10 +0800
    Ready:          True
    Restart Count:  0
    Limits:
      cpu:     500m
      memory:  256Mi
    Requests:
      cpu:        50m
      memory:     128Mi
    Environment:  <none>
    Mounts:
      /otel-auto-instrumentation-nodejs from opentelemetry-auto-instrumentation-nodejs (rw)
      /var/run/secrets/kubernetes.io/serviceaccount from kube-api-access-2vf2n (ro)
```

A sidecar container is added to the application pod to run the instrumentation library

© Copyright National University of Singapore

Example - Auto Injection Setup

```
Containers:
  bggapp-container:
    Container ID:   docker://7ca3a26faf70333246ca27bcae5fb353f36bc4c849ec8b62a66358d543702ba1
    Image:          chukmunnlee/bgg-app:nov-2025
    Image ID:       docker-pullable://chukmunnlee/bgg-app@sha256:4699ed534a33aa108f399ea6c6674ebabef47cc59653fd46002d905d719b5f75
    Port:           5000/TCP
    Host Port:      0/TCP
    State:          Running
      Started:      Mon, 01 Dec 2025 09:14:11 +0800
    Ready:          True
    Restart Count:  0
    Environment Variables from:
      bgg-secret  Secret     Optional: false
      bgg-cm      ConfigMap  Optional: false
    Environment:
      OTEL_NODE_IP:                          (v1:status.hostIP)
      OTEL_POD_IP:                           (v1:status.podIP)
      NODE_OPTIONS:                          --require /otel-auto-instrumentation-nodejs/autoinstrumentation.js
      OTEL_SERVICE_NAME:                     bggapp-deploy
      OTEL_EXPORTER_OTLP_ENDPOINT:           http://bgg-otelcol-collector.bggns.svc.cluster.local:4318
      OTEL_RESOURCE_ATTRIBUTES_POD_NAME:     bggapp-deploy-b74d55f55-62wjq (v1:metadata.name)
      OTEL_RESOURCE_ATTRIBUTES_NODE_NAME:    (v1:spec.nodeName)
      OTEL_PROPAGATORS:                      tracecontext,baggage
      OTEL_TRACES_SAMPLER:                   parentbased_traceidratio
      OTEL_TRACES_SAMPLER_ARG:               1
      OTEL_RESOURCE_ATTRIBUTES:              k8s.container.name=bggapp-container,k8s.deployment.name=bggapp-deploy,k8s.namespace.name=bggns,k8s.node.name=$(OTEL_RESOURCE_ATTRIBUTES_NODE_NAME),k8s.pod.name=$(OTEL_RE
SOURCE_ATTRIBUTES_POD_NAME),k8s.replicaset.name=bggapp-deploy-b74d55f55,service.instance.id=bggns.$(OTEL_RESOURCE_ATTRIBUTES_POD_NAME).bggapp-container,service.namespace=bggns,service.version=nov-2025
    Mounts:
      /otel-auto-instrumentation-nodejs from opentelemetry-auto-instrumentation-nodejs (rw)
      /var/run/secrets/kubernetes.io/serviceaccount from kube-api-access-2vf2n (ro)
```

Environment variables are injected into the application container to initialize and start auto instrumentation
Language runtime specific

© Copyright National University of Singapore

Push and Pull Options for Metrics

© Copyright National University of Singapore

Prometheus Push and Pull Metrics Collection

- Pull is the default way Prometheus collects metrics
  - Prometheus actively pulls metrics from the application
    - Either statically configured or dynamically discovered
- Metrics can be pushed to Prometheus if remote write endpoint is enabled
  - Off by default, enabled with `--enable-feature` option
  - `remote-write-receiver` - pushes metric to Prometheus with Prometheus Remote Wire Protocol
    - `POST /api/v1/write`
  - `otlp-write-receiver` - pushes metric to Prometheus with OTLP (OpenTelemetry Protocol)
    - `POST /api/v1/otlp/v1/metrics`

```
--enable-feature=otlp-write-receiver,remote-write-receiver
```

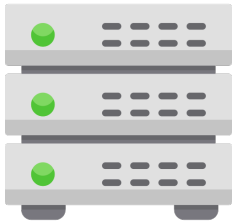
© Copyright National University of Singapore

OpenTelemetry Push and Pull Metrics Collection

- OpenTelemetry can both push and pull metrics on the receivers and exporters configuration
- Receiver
  - `otlp` - metrics are pushed to the collector using the OTLP
  - `prometheus` - metrics are pull by the collector with the Prometheus wire format
- Exporter
  - `otlphttp` - metrics are push to Prometheus in OTLP
  - `prometheus` - metrics are pulled from scrape targets, behaves like Prometheus
  - `promethusremotewrite` - metrics are push to Prometheus in Prometheus wire format

© Copyright National University of Singapore

OpenTelemetry Push and Pull Metrics Collection



Receiver	Exporter
otlp	otlphttp
	prometheusr emotewrite
prometheus	prometheus

© Copyright National University of Singapore

OpenTelemetry Metrics Receivers

	`prometheus`	`otlp`
Model	Pull	Push
Transport	Prometheus exposition format	OTLP either with gRPC or HTTP
Initiator	Collector	Application
Metrics protocol	Prometheus	OpenTelemetry
Latency	Depends on scrape interval	Near real-time

© Copyright National University of Singapore

OpenTelemetry Metrics Exporters

	`prometheus`	`prometheusremotewrite`	`otlphttp`
Model	Pull	Push	Push
Protocol	Prometheus exposition format	Prometheus remote write	OTLP over HTTP
Initiator	Prometheus	Collector	Collector
OTel metadata preserved	Limited	Limited	Best
Latency	Depends on scrape interval	Near real-time	Near real-time
Prometheus feature flag	Configure Prometheus to scrape endpoint	`remote-write-receiver`	`otlp-write-receiver`

© Copyright National University of Singapore

Example - `prometheusremotewrite`

Set `--enable-feature=remote-write-receiver` option

Set `--enable-feature=otlp-write-receiver` option

```
receivers:
  otlp:
    protocols:
      grpc:
        endpoint: 0.0.0.0:4317
      http:
        endpoint: 0.0.0.0:4318
exporters:
  prometheusremotewrite/prom-0:
    endpoint: http://prometheus-0:9090/api/v1/write
  otlphttp/prom-1:
    endpoint: http://prometheus-1:9090

service:
  pipelines:
    metrics:
      receivers: [ otlp ]
      exporters: [ prometheusremotewrite/prom-0 ]
```

© Copyright National University of Singapore

Workshop 4
Task 4

© Copyright National University of Singapore

Prometheus

© Copyright National University of Singapore

OpenTelemetry Prometheus Receiver

- Need to configure scape targets when configuring the Prometheus receiver
- **`static_configs`** - specify a list of targets
  - Used in environments where the applications are fairly static
    - Eg. Applications deployed in servers
  - Not ideal for containerized applications running in a dynamic environment like Kubernetes, service meshes, container runtime

Statically configured scrape targets

```
receivers:
  prometheus:
    scrape_configs:
    - job_name: books
      static_configs:
      - targets: [ 'checkout-svc:5000' ]
        labels:
          env: prod
      - targets: [ 'cart-svc:5000' ]
        labels:
          env: prod
```

© Copyright National University of Singapore

Prometheus Service Discovery

- Mechanism that allows Prometheus to query a controller to automatically discover what applications are running
  - Controller is typically a control plane component like API server in Kubernetes, Docker Engine, cloud providers, etc.
- Service discovery operation stages
  - Discovery what applications are running by querying the controller
  - Controller returns metadata on the resource eg. pods, services, nodes that can potentially be used to identify the metrics produced by these resources
    - Metadata starts with `__meta_`
    - `__address__` is the endpoint used for scraping the metrics, defaults to the container port
      - Format `<pod_ip>:<container_port>` format
  - Relabel performs a series of steps to decide which of the metadata to keep
    - Select and rewrite the metadata, any metadata not selected will be dropped
    - Change `__address__` if necessary

© Copyright National University of Singapore

Kubernetes Service Discovery

- Kubernetes service discovery can discover the following resource called role
  - node - Kubernetes nodes
  - pod - services running in containers
  - service - a group of pods
  - ingress - point in which external traffic enters the cluster
- Metadata are constructed from
  - Resource type
    - Eg. pod name, pod IP, service namespace, service cluster IP, node OS, node container runtime version, etc
  - Labels and annotations
  - List of scraped metadata
    - https://prometheus.io/docs/prometheus/latest/configuration/configuration/#kubernetes_sd_config

© Copyright National University of Singapore

Example - Metadata

Application produce the following metrics

`http_request_total`

Which of these to include as labels

```
__meta_kubernetes_pod_name = dov-bear
__meta_kubernetes_pod_namespace = prod
__meta_kubernetes_pod_ip = <assigned pod IP>
__meta_kubernetes_pod_ready = <true/false>
__meta_kubernetes_pod_phase = <Pending/Running/...>
__meta_kubernetes_pod_node_name = <node name scheduled on>
__meta_kubernetes_pod_host_ip = <node IP>
__meta_kubernetes_pod_label_app = dov
__meta_kubernetes_pod_label_tier = backend
__meta_kubernetes_pod_annotation_prometheus_io_scrape = true
__meta_kubernetes_pod_annotation_prometheus_io_port = 3000
```

```yaml
apiVersion: v1
kind: Pod
metadata:
  name: dov-bear
  namespace: prod
  labels:
    app.kubernetes.io/name: dov
    tier: backend
  annotations:
    prometheus.io/scrape: 'true'
    prometheus.io/port: '3000'
```

© Copyright National University of Singapore

Relabel Config

- Relabel config specifies how metadata from resources are processed sequentially in a pipeline
- `source_label` specifies a list of metadata to consider in an array
  - Metadata values are concatenated with semi-colon (`;`)
- `action` defines what actions to perform, common actions
  - `keep`, `drop` - keep or drop a target if the value does not match the `regex`
  - `replace` - sets the `target_label` based on the regex `value`

```
source_labels:
- metadata_0
- metadata_1
action: <action to perform>
regex: <label value to match>
replacement: <value to save>
target_label: <label to save replacement>
```

Concatenated as `value_1;value_2`

- `regex` - either used for matching or destructuring values
  - Depending on the action
- `replacement` - create a new label value from `regex` capture group
- `target_label` - label name that the value from replacement is written to

© Copyright National University of Singapore

Example - Scraping Pods

1. If the `prometheus.io/scrape` annotation is 'true', continue to the next relabel action; otherwise drop the target if it is not or missing
2. If the `prometheus.io/port` annotation exists replace the port in the `__address__` label
3. Create a new label `service_name` from the value of the pod label `app.kubernetes.io/name`

Discover running pods

Pod metadata pipeline

```
receivers:
  prometheus:
    config:
      scrape_configs:
      - job_name: k8s-pods
        kubernetes_sd_configs:
        - role: pod
        relabel_configs:
        - source_labels:[ __meta_kubernetes_pod_annotation_prometheus_io_scrape ]
          action: keep
          regex: true
        - source_labels: [__address__, __meta_kubernetes_pod_annotation_prometheus_io_port ]
          action: replace
          regex: (.+):(d+);(\d+)
          replacement: $1:$3
          target_label: __address__
        - source_labels:[ __meta_kubernetes_pod_label_app_kubernetes_io_name ]
          action: replace
          regex: (.+)
          replacement: $1
          target_label: service_name
```

© Copyright National University of Singapore

Example - Scraping Pods

```
kubernetes_sd_configs:
- role: pod
```

Scrape pods

```
labels:
  app.kubernetes.io/name: dov
annotations:
  prometheus.io/scrape: 'true'
  prometheus.io/port: '3000'
```

Produce metadata from pod resources

```
__address__: 10.244.10.1:8080
__metadata_kubernetes_pod_annotations_prometheus_io_scrape: true
__metadata_kubernetes_pod_annotations_prometheus_io_port: 3000
__metadata_kubernetes_pod_labels_app_kubernetes_io_name: dov
```

```
relabel_configs:
- source_labels: [ __meta_... ]
...
```

Feed metadata into relabel pipeline

```
__address__: 10.244.10.1:3000
service_name: dov
...
```

Filter targets and select metadata as labels

```
up{service_name="dov" ... }
http_request_total{service_name="dov" ... }
```

Label metrics from pod with the labels

© Copyright National University of Singapore

Prometheus Service Discovery

Prometheus | Query | Alerts | Status > Service discovery

serviceMonitor/monitoring/prom-stack-grafana/0 1 / 24

Discovered labels

- __address__="10.244.0.23:3000"
- __meta_kubernetes_endpoint_address_target_kind="Pod"
- __meta_kubernetes_endpoint_address_target_name="prom-stack-grafana-8579555cdc-6mg6q"
- __meta_kubernetes_endpoint_node_name="minikube"
- __meta_kubernetes_endpoint_port_name="http-web"
- __meta_kubernetes_endpoint_port_protocol="TCP"
- __meta_kubernetes_endpoint_ready="true"
- __meta_kubernetes_endpoints_annotation_endpoints_kubernetes_io_last_change_trigger_time="2026-02-25T23:29:21Z"
- __meta_kubernetes_endpoints_annotationpresent_endpoints_kubernetes_io_last_change_trigger_time="true"
- __meta_kubernetes_endpoints_label_app_kubernetes_io_instance="prom-stack"
- __meta_kubernetes_endpoints_label_app_kubernetes_io_managed_by="Helm"
- __meta_kubernetes_endpoints_label_app_kubernetes_io_name="grafana"
- __meta_kubernetes_endpoints_label_app_kubernetes_io_version="12.4.0"
- __meta_kubernetes_endpoints_label_endpoints_kubernetes_io_managed_by="endpoint-controller"
- __meta_kubernetes_endpoints_label_helm_sh_chart="grafana-11.2.0"
- __meta_kubernetes_endpoints_labelpresent_app_kubernetes_io_instance="true"
- __meta_kubernetes_endpoints_labelpresent_app_kubernetes_io_managed_by="true"
- __meta_kubernetes_endpoints_labelpresent_app_kubernetes_io_name="true"
- __meta_kubernetes_endpoints_labelpresent_app_kubernetes_io_version="true"
- __meta_kubernetes_endpoints_labelpresent_endpoints_kubernetes_io_managed_by="true"
- __meta_kubernetes_endpoints_labelpresent_helm_sh_chart="true"
- __meta_kubernetes_endpoints_name="prom-stack-grafana"
- __meta_kubernetes_namespace="monitoring"

Target labels

- container="grafana"
- endpoint="http-web"
- instance="10.244.0.23:3000"
- job="prom-stack-grafana"
- namespace="monitoring"
- pod="prom-stack-grafana-8579555cdc-6mg6q"
- service="prom-stack-grafana"

show relabeling

© Copyright National University of Singapore

Workshop 4
Task 5

© Copyright National University of Singapore

End of Module

© Copyright National University of Singapore